

Beginner-C Bootcamp 2주차 과제

20215789 산업보안학과 김문정

Challenge 1. 자료형 크기 확인하기

```
user@user-VirtualBox:~$ vim sizeof.c
user@user-VirtualBox:~$ gcc sizeof.c -o sizeof
sizeof.c: In function 'main':
sizeof.c:5:15: warning: format '%d' expects argument of type 'int', but argument
 2 has type 'long unsigned int' [-Wformat=]
   5 | printf("char:%d",sizeof(v_char));
     |               ^~
     |               |
     |               | long unsigned int
     |               int
     |               %ld
user@user-VirtualBox:~$
```

처음에는 시험삼아 코드를

```
char v_char ='a';
```

```
printf("char:%d", sizeof(v_char));
```

로 작성하였으나 화면과 같이 에러가 발생하는 것을 알 수 있었다. 이유를 알아보니 %d는 부호 있는 10진 정수를 나타내는 서식지정자인데, sizeof()의 형태는 보통 unsigned long타입이어서 경고가 뜨는 것을 확인할 수 있었다.

따라서 서식지정자를 %zu로 바꾸고 다시 컴파일하였더니

```
user@user-VirtualBox:~$ gcc sizeof.c -o sizeof
sizeof.c:32:1: error: expected identifier or '(' before '}' token
   32 | }
     | ^
user@user-VirtualBox:~$ vim sizeof.c
user@user-VirtualBox:~$ gcc sizeof.c -o sizeof
user@user-VirtualBox:~$ ./sizeof
=== 기본 자료형 크기 ===
char: 1 byte
short: 2 byte
int: 4 byte
long: 8 byte
long long: 8 byte
float: 4 byte
double: 8 byte

=== signed / unsigned 비교 ===
signed char: 1 byte
unsigned char: 1 byte
signed int: 4 byte
unsigned int: 4 byte
signed long: 8 byte
unsigned long: 8 byte
user@user-VirtualBox:~$
```

와 같은 결과가 나오는 것을 확인하였다.

자료형			설명	바이트	범위
정수형	부호있음	short	short형 정수	2	-32768 ~ 32767 ☆ 크기
		int ☆ / d (변경)	정수	4	-2147483648 ~ 2147483647
		long	long형 정수	4	-2147483648 ~ 2147483647
	부호없음	unsigned short	부호없는 short형 정수	2	0 ~ 65535
		unsigned int	부호없는 정수	4	0 ~ 4294967295
		unsigned long	부호없는 long형 정수	4	0 ~ 4294967295
문자형	부호있음	char ☆ / c (변경)	문자 및 정수	1	-128 ~ 127 ☆ 크기
	부호없음	unsigned char	문자 및 부호없는 정수	1	0 ~ 255 ☆
부동소수점형 (실수)		float	단일정밀도 부동소수점	4	1.2E-38 ~ 3.4E38
		double ☆ / f (변경)	두배정밀도 부동소수점	8	2.2E-308 ~ 1.8E308

출력 결과는 위 사진과 같고, signed와 unsigned는 부호있음/없음의 차이로 char의 범위가 -128~127이라면, unsigned char는 128+127인 0~255까지의 범위를 뜻함을 알 수 있었다.

자료형의 크기가 이렇게 정해지는 이유는 CPU가 한번에 처리하기 가장 효율적인 크기로 정해지고, 운영체제나 데이터 모델에 따라 달라지며 IEEE 754같은 표준에 의해 정해지기 때문이라는 사실도 알 수 있었다.

Challenge 2. 비트 연산 프로그램 작성

```

user@user-VirtualBox: ~
user@user-VirtualBox:~$ vim bitbit.c
user@user-VirtualBox:~$ gcc bitbit.c -o bitbit
user@user-VirtualBox:~$ ./bitbit
bash: ./bitbit: No such file or directory
user@user-VirtualBox:~$ ./bitbit
enter two integer:1 3
AND: 1
OR:3
XOR:2
a NOT:-2
a LEFT SHIFT:2
b RIGHT SHIFT:1
user@user-VirtualBox:~$

```

- 비트연산자

비트 연산자는 비트(bit) 단위로 논리 연산을 할 때 사용하는 연산자입니다.

또한, 비트 단위로 전체 비트를 왼쪽(<<)이나 오른쪽(>>)으로 이동시킬 때도 사용합니다.

연산자	의미
&	and 모든 비트가 1일 때만 1
^	xor 모든 비트가 같으면 0, 하나라도 다르면 1
	or 모든 비트 중 한 비트라도 1이면 1
~	not 각 비트의 부정, 0이면 1, 1이면 0
<<	왼쪽 시프트 비트를 왼쪽으로 이동 $\times 2^n$
>>	오른쪽 시프트 비트를 오른쪽으로 이동 $\div 2^n$

우선 이진수로 말고 십진수로 출력되도록 프로그램을 작성했다. (맨 뒷사진 참고) 그러나 이 연산의 결과를 어떻게 이진수로 출력할 수 있을지 몰라 AI의 도움을 받았다. 아이디어는 숫자는 내부적으로 이미 2진수로 저장되어 있으므로, 그것을 한 자리씩 꺼내어 출력하는 코드를 작성하면 된다는 것이었다.

코드는 다음과 같다.

```
void print_binary(int num) {  
    for (int i = 7; i >= 0; i--) { // 8비트  
        printf("%d", (num >> i) & 1);  
    }  
}
```

i번째 비트가 0인지 1인지 알기 위하여 위 코드를 작성한 것을 알 수 있다. 예를 들어, num=1일 경우 (i=3부터 시작이라고 가정) i=3일 때 0001중 맨 앞 비트(가장 왼쪽_0)를 가장 끝(오른쪽)으로 보내기 위해 >>연산을 하고, 그렇게 온 0을 0/1을 판단하기 위해 1과 &연산을 하는 방식으로 동작한다는 것을 알 수 있었다. for문을 돌려 0000 0000 형태의 8비트 형태로 출력하면 이진수로 출력된다는 것 또한 알 수 있었다.

이제 이진수까지 출력해보면

```
user@user-VirtualBox:~$ vim bitbit.c
user@user-VirtualBox:~$ vim bitbit.c
user@user-VirtualBox:~$ gcc bitbit.c -o bitbit
user@user-VirtualBox:~$ ./bitbit
enter two integer:1 3
AND: 1
0000 0001
OR:3
0000 0011
XOR:2
0000 0010
a NOT:-2
1111 1110
a LEFT SHIFT:2
0000 0010
b RIGHT SHIFT:1
0000 0001
```

추가적으로 왜 a NOT이 십진수로 -2라고 출력되는지 조사해보았다. (1111 1110은 254인데 왜 -2라고 표현되는지 궁금했다.)

-> 만약 unsigned char였으면 부호가 없으므로 254로 해석하는게 맞는데, C에서 기본 자료형은 signed로 보기 때문에 MSB가 1일 경우 음수로 해석한다는 것을 알 수 있었다. 우선 모든 비트를 반전하고, 0000 0001 거기에 1을 더해 0000 0010으로 2의 보수 방식으로 표현한다는 것을 알 수 있었다.

Challenge3. Integer Overflow

```
user@user-VirtualBox:~$ ^C
user@user-VirtualBox:~$ vim limit.c
user@user-VirtualBox:~$ gcc limit.c -o limit
user@user-VirtualBox:~$ ./limit
Signed INT_MAX: 2147483647
Signed INT_MAX + 1: -2147483648
Unsigned UINT_MAX: 4294967295
Unsigned UINT_MAX + 1: 0
user@user-VirtualBox:~$
```

int의 범위는 -2,147,483,648 ~ 2,147,483,647이고, 최댓값에서 1을 더해버리면 0111 1111 1111 1111 1111..... +1을 하기 때문에 오버플로우가 발생한다는 것을 알 수 있다. int는 signed 로 들어가기 때문에, 2번문제에서 조사한 것처럼 MSB가 음수가 되므로 음수로 해석하게 되어 가장 낮은 숫자인 -2147483648이 된다는 것을 알 수 있다. 반면 unsigned의 범위는 0 ~ 4,294,967,295 이고, 범위를 넘어 연산을 하였을 경우 mod연산

이 적용되어 최댓값 $+1 = 4294967295 + 1 = 4294967296$ 을 $2^{32} = 4294967296$ 로 나눈 나머지 $= 0$ 이므로 최댓값을 넘어가면 0부터 다시 시작하는 순환 연산을 한다는 것을 알 수 있다. (overflow가 생겨도 음수로 갈 수 없음)